

Object Oriented Systems (OOS) — CO4 Complete Detailed Solutions

Question 4 (a), (b), and (c)

Based on the uploaded university examination paper.

Introduction

Question 4 focuses on one of the most important software engineering and system design technologies:

UML — Unified Modeling Language

UML is a standardized visual modeling language used for:

-  software system design
-  object-oriented analysis
-  system visualization
-  architecture planning
-  communication between developers and clients

Before coding a large software system, engineers first create UML models to understand requirements, design architecture, define interactions, and avoid implementation errors. UML acts as the blueprint of software systems exactly like architectural drawings for buildings. 

This question discusses:

-  Aggregation vs Composition
-  Include vs Extend Relationship
-  Use Case vs Sequence Diagram
-  Access Specifiers in UML
-  Multiplicity

These concepts form the backbone of software modeling and object-oriented system analysis.

Question 4(a)(i)

Exact Question

Distinguish between Aggregation and Composition relation with proper examples.

Introduction to Relationships in UML

In object-oriented systems, classes rarely exist independently. Objects interact and establish relationships with each other. Among these relationships, Aggregation and Composition are extremely important because they describe **HAS-A** relationships between objects.

What Is Aggregation?

Aggregation represents a **weak HAS-A relationship**. In aggregation:

- ☒ child object can exist independently
- ☒ parent object does not own child completely
- ☒ lifecycle dependency is weak

What Is Composition?

Composition represents a **strong HAS-A relationship**. In composition:

- ☒ child object cannot exist independently
- ☒ parent completely owns child
- ☒ lifecycle dependency is strong

Aggregation vs Composition

Feature	Aggregation	Composition
Relationship Strength	Weak	Strong
Ownership	Partial	Complete
Child Independence	Possible	Not Possible
Lifecycle Dependency	Weak	Strong
UML Symbol	Hollow Diamond	Filled Diamond
Example	Department–Teacher	House–Room

Aggregation Example

 **Real-World Example** A Department has Teachers. However, a teacher can exist without a department, and a teacher may transfer to another department. Thus the relationship is weak.

Java Example — Aggregation

```
class Teacher {
    String name;

    Teacher(String name) {
        this.name = name;
    }
}

class Department {
    String deptName;
    Teacher teacher;

    Department(String deptName, Teacher teacher) {
        this.deptName = deptName;
        this.teacher = teacher;
    }
}
```

```
    }

    void display() {
        System.out.println(teacher.name + " works in " + deptName);
    }
}

class Demo {
    public static void main(String args[]) {
        Teacher t = new Teacher("Arjun");
        Department d = new Department("CSE", t);
        d.display();
    }
}
```

Output

```
Arjun works in CSE
```

 **Why This Is Aggregation** The **Teacher** object exists independently. Even if the **Department** object is destroyed, the **Teacher** object may still exist. Thus ownership is weak.

Composition Example

 **Real-World Example** A House contains Rooms. If the house is destroyed, the rooms cannot exist independently. Thus the relationship becomes strong.

Java Example — Composition

```
class Room {
    Room() {
        System.out.println("Room Created");
    }
}

class House {
    private Room room;

    House() {
        room = new Room(); // Tightly coupled lifecycle
    }
}

class Demo {
    public static void main(String args[]) {
        House h = new House();
    }
}
```

```
}
}
```

Output

Room Created

 **Why This Is Composition** The **Room** object is created inside **House**. Its lifecycle depends entirely on **House**. Thus ownership becomes strong.

UML Representation

Aggregation (Hollow Diamond)

Department ◇—— Teacher

Composition (Filled Diamond)

House ◆—— Room

Real-World Intuition

- **Aggregation:**  University has Professors.
- **Composition:**  Car has Engine. Without the car, the engine relationship becomes meaningless in that context.

 **Conclusion** Aggregation models weak ownership whereas composition models strong ownership with lifecycle dependency.

Question 4(a)(ii)

Exact Question

Distinguish between include and extend relation.

Introduction

Use Case Diagrams model system functionality from the user's perspective. Two important relationships in use cases are Include and Extend. They help organize reusable and optional behaviors.

Include Relationship

Include means **one use case compulsorily uses another use case**. It represents mandatory reusable functionality.

🗨️ Extend Relationship

Extend means **one use case optionally adds behavior to another**. It represents conditional or optional functionality.

📊 Include vs Extend

Feature	Include	Extend
Nature	Mandatory	Optional
Purpose	Reuse common functionality	Add conditional behavior
Dependency	Strong	Weak
Arrow Direction	Base → Included	Extension → Base
Execution	Always executes	Executes conditionally

🌟 Include Example

📘 **ATM System** To withdraw money, a user must authenticate. Thus:

```
Withdraw Cash ----<<include>>---- Authenticate User
```

Authentication always occurs.

🌟 Extend Example

📘 **Online Shopping** During payment, a customer may optionally apply a coupon. Thus:

```
Payment ----<<extend>>---- Apply Coupon
```

Coupon usage is optional.

🗨️ Deep Conceptual Analysis

- **Why Include Exists:** Without include, we duplicate common behavior repeatedly. Include improves reusability, maintainability, and modularity.
- **Why Extend Exists:** Extend supports optional scenarios. Real systems often contain alternate paths, exception cases, and conditional actions. Extend models them elegantly.

💡 Real-World Intuition

- **Include:** 📄 Every restaurant bill must calculate tax.
- **Extend:** 🎁 A festival discount may optionally apply.

🎯 **Conclusion** Include represents mandatory reusable functionality whereas extend represents optional conditional functionality.



Question 4(a)(iii)



Exact Question

Distinguish between Use Case Diagram and Sequence Diagram.



Introduction

UML diagrams are broadly categorized into Structural diagrams and Behavioral diagrams. Both Use Case and Sequence diagrams describe behavior, but from different perspectives.



Use Case Diagram

A Use Case Diagram shows system functionalities, actors interacting with the system, and external user perspective.

- **It focuses on:** WHAT the system does.



Sequence Diagram

A Sequence Diagram shows interaction flow between objects, order of message passing, and execution sequence over time.

- **It focuses on:** HOW the system works internally.



Use Case vs Sequence Diagram

Feature	Use Case Diagram	Sequence Diagram
Focus	Functional requirements	Interaction flow
Perspective	External	Internal
Shows	Actors and use cases	Object communication
Time Sequence	Not shown	Explicitly shown
Complexity	Simpler	More detailed
Main Purpose	Requirement analysis	Dynamic behavior modeling



Use Case Example

```
Customer ---- Place Order
Customer ---- Make Payment
```

Shows system services.



Sequence Diagram Example

Customer → OrderSystem → PaymentGateway

Shows interaction sequence.

🗨️ Deep Analysis

- **Why Use Case Diagrams Matter:** Before coding begins, clients describe requirements, and analysts identify functionalities. Use Case diagrams help bridge the communication gap.
- **Why Sequence Diagrams Matter:** During implementation, developers understand message flow, execution order becomes clear, and object collaboration is visualized.

💡 Real-World Intuition

- **Use Case Diagram:** 📄 Restaurant menu.
- **Sequence Diagram:** 👨‍🍳 Actual cooking process in the kitchen.

🎯 **Conclusion** Use Case diagrams describe system functionality externally while Sequence diagrams describe internal object interactions over time.

🚀 Question 4(b)

📘 Exact Question

State how the different access specifiers of member methods are represented in a class diagram.

📘 Introduction

Class diagrams represent classes, attributes, methods, and relationships. UML uses special symbols to represent visibility/accessibility. These symbols correspond directly to Java access modifiers.

🎨 UML Access Specifier Symbols

Access Specifier	UML Symbol	Meaning
Public	+	Accessible everywhere
Private	-	Accessible only inside class
Protected	#	Accessible in subclasses
Default/Package	~	Accessible within package

🌟 UML Class Diagram Example

```
-----  
|           Student           |  
-----  
| - rollNo : int              |  
| # marks : float             |
```

```

| ~ address : String      |
|-----|
| + display() : void      |
| - calculate() : int     |
|-----|

```

💡 Real-World Intuition

Imagine a company:

- **Public (+):** Open to everyone.
- **Protected (#):** Available to team members.
- **Private (-):** Personal confidential data.
- **Default (~):** Department-only access.

🔗 **Conclusion** UML class diagrams use symbolic visibility notation to represent encapsulation and access control mechanisms clearly.

🚀 Question 4(c)

📘 Exact Question

Discuss multiplicity with suitable example.

📘 Introduction

Multiplicity specifies: 🔥 **How many objects participate in a relationship.** It defines object cardinality. Multiplicity is extremely important in database modeling and object-oriented design.

💡 Why Multiplicity Is Needed

Without multiplicity, relationship quantity becomes ambiguous. Multiplicity clarifies one-to-one, one-to-many, many-to-many, and optional participation.

📊 Common Multiplicity Values

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one
*	Many
1..*	One or many
0..*	Zero or many

✨ Examples

📘 Example 1 — One-to-One

Person 1 ----- 1 Passport

One person has one passport.

Example 2 — One-to-Many

Department 1 ----- 1..* Student

One department contains many students.

Example 3 — Many-to-Many

Student * ----- * Course

One student can enroll in many courses and one course can contain many students.

Java Example (One-to-Many)

```
class Student {
    String name;
    Student(String name) {
        this.name = name;
    }
}

class Department {
    Student students[];

    Department(Student students[]) {
        this.students = students;
    }

    void display() {
        for(Student s : students) {
            System.out.println(s.name);
        }
    }
}

class Demo {
    public static void main(String args[]) {
        Student arr[] = {
            new Student("Amit"),
            new Student("Rahul"),
            new Student("Sneha")
        };
    }
}
```

```
    Department d = new Department(arr);  
    d.display();  
}  
}
```

Output

```
Amit  
Rahul  
Sneha
```

 **Why This Demonstrates Multiplicity** One **Department** object contains multiple **Student** objects. Thus multiplicity becomes: **1** → **many**.

Deep Conceptual Analysis

Multiplicity helps in database schema design, object relationship clarity, software architecture planning, and validation of business rules.

Real-World Intuition

Multiplicity is like counting relationships:

-  One teacher teaches many students.
-  One school has many classrooms.
-  One user owns multiple devices.

Final Conclusion

Question 4 introduces fundamental UML modeling concepts essential for software engineering and object-oriented analysis. The discussed concepts include Aggregation and Composition, Include and Extend Relationships, Use Case and Sequence Diagrams, Access Specifier Representation, and Multiplicity. Together, these concepts help developers design scalable, maintainable, modular, and well-structured software systems before implementation begins.   